

Guion — Microservicio de Órdenes de Compra

Duración sugerida: 12-15 minutos. Puede dividirse en 2 videos si es necesario.

Bloque naranja = qué hacer / dónde hacer clic. **Bloque azul** = qué decir en voz alta.

0. Prepara tus pestañas antes de grabar

QUÉ HACER (antes de grabar):

1. Abre 2 pestañas del navegador: (1) tu sitio de Netlify, (2) MongoDB Atlas ya con sesión iniciada en tu cluster.
2. Abre VS Code con el proyecto, y deja abiertos en pestañas: `Models/PurchaseOrder.cs` , `Data/MongoOrderRepository.cs` , `Application/OrderService.cs` , `Endpoints/OrdersEndpoint.cs` , `Program.cs` , `frontend/src/App.vue` .
3. En Netlify, agrega ya 1 producto al carrito.

1. Introducción (30 s)

QUÉ HACER: Muestra el editor con el proyecto abierto (carpeta `eshop-services` visible). No hace falta hacer clic todavía.

"Buenas, en este video voy a explicar la segunda fase del proyecto: la implementación de un tercer microservicio, Order.API, encargado de generar órdenes de compra a partir del carrito. A diferencia de Catalog.API y Basket.API, que persisten en PostgreSQL, este microservicio usa MongoDB Atlas como base de datos documental. Voy a explicar la arquitectura del código, y luego una demostración completa: desde agregar productos al carrito hasta consultar la orden generada, cambiarle el estado, descargarla en PDF, y verificar la persistencia directamente en MongoDB Atlas."

2. Arquitectura y código (4-5 min)

2.1 Vista general

QUÉ HACER: En el explorador de VS Code, expande `src/Order.API` para ver `Models` , `Data` , `Application` , `Endpoints` , `Services` .

"El microservicio sigue el mismo patrón que Catalog.API y Basket.API: ASP.NET Core Minimal API con Carter para el enrutamiento, pero con una separación clara de responsabilidades en cuatro capas."

2.2 Modelo de dominio

QUÉ HACER: Abre `Models/PurchaseOrder.cs` , señala `Id` , `CustomerId` , `CreatedAt` , `Status` , `Items` , `Subtotal` , `Tax` , `Total` . Cambia a `Models/OrderItem.cs` , señala `ProductId` , `Quantity` , `UnitPrice` .

"Primero, el dominio: la clase `PurchaseOrder` representa la orden, con un identificador único, el cliente, la fecha de creación, el estado, la lista de `OrderItem` , y los totales: subtotal, impuesto y total. Cada `OrderItem` guarda el producto, la cantidad, y el precio unitario congelado al momento de la compra — no se recalcula después, aunque el precio del catálogo cambie."

2.3 Persistencia (MongoDB)

QUÉ HACER: Abre `Data/MongoOrderRepository.cs` . Señala el método `RunAsync` (línea ~22-34), luego el `catch (Exception ex) when (ex is MongoException or TimeoutException)` , y el constructor con `MongoClient(...)` .

"La capa de persistencia usa el driver oficial de MongoDB para .NET. Cada operación pasa por un método `RunAsync` que envuelve las llamadas al driver: si MongoDB no está disponible o hay un timeout, se captura la excepción interna del driver y se traduce a un error genérico. Esto es importante porque el driver por defecto expone detalles internos del clúster en el mensaje de error, y eso viola el requisito de no exponer información sensible al cliente. Aquí también hay un detalle técnico relevante: el driver de MongoDB en su versión 3 requiere declarar explícitamente cómo serializar los `Guid` , cosa que se configura una sola vez al inicio de `Program.cs` ."

2.4 Lógica de negocio

QUÉ HACER: Abre `Application/OrderService.cs` . Señala `AllowedTransitions` (línea ~25-29). En `CreateOrderAsync` , señala en orden: el bloque de `idempotencyKey` , la llamada a `basketClient.GetBasketAsync` , y el `foreach` con `catalogClient.ProductExistsAsync` .

"La capa de aplicación es donde vive la lógica de negocio. Al crear una orden, primero se revisa si ya existe una orden con el mismo `Idempotency-Key` para ese cliente — si existe, se devuelve esa misma orden en lugar de crear una nueva. Esto implementa idempotencia: si el cliente reintenta la misma solicitud, por ejemplo por un error de red, no se genera una compra duplicada. Después, se obtiene el carrito llamando por HTTP a `Basket.API`, se valida que no esté vacío, y por cada producto se valida que exista consultando a `Catalog.API`. Si algo falla en cualquiera de estas validaciones, se lanza una excepción que se traduce a `400 Bad Request` . Aquí también está la máquina de estados: un diccionario define las transiciones válidas — de `Pending` se puede pasar a `Confirmed` o a `Cancelled` , pero desde esos dos estados finales no se puede transicionar a ningún otro."

2.5 Endpoints

QUÉ HACER: Abre `Endpoints/OrdersEndpoint.cs` . Señala de arriba a abajo cada `group.MapPost/MapGet/MapPatch` .

"Finalmente, los endpoints: `POST /api/orders` para crear, `GET /api/orders` para listar todas, `GET /api/orders/{id}` para consultar una específica, `GET /api/orders/customer/{customerId}` para las de un cliente, y `PATCH /api/orders/{id}/status` para cambiar el estado. Todo documentado con Swagger."

2.6 Ensamblado del servicio (Program.cs)

QUÉ HACER: Abre `Program.cs` . Señala en orden: `BsonSerializer.RegisterSerializer(...)` , `AddSingleton<IOrderRepository, MongoOrderRepository>()` , los dos `AddHttpClient` (`Basket` y `Catalog`), `AddExceptionHandler<CustomExceptionHandler>()` , y `AddSwaggerGen()` / `UseSwaggerUI()` .

"Todo esto se ensambla en `Program.cs` , el punto de entrada del microservicio. Aquí se registra el repositorio de MongoDB como singleton, y dos clientes HTTP tipados — `IBasketApiClient` y `ICatalogApiClient` — apuntando a las URLs de los otros microservicios, configuradas por variables de entorno. También se reutiliza `CustomExceptionHandler` , el mismo manejador de errores centralizado que ya usan `Catalog.API` y `Basket.API`, para mantener consistencia en cómo se devuelven los errores en toda la solución. Y se habilita Swagger, que genera automáticamente la documentación interactiva de la API a partir de estos mismos endpoints."

2.7 Frontend

QUÉ HACER: Abre `frontend/src/App.vue`. Scroll rápido mostrando el `cart-drawer`, la vista `orders` y la vista `order` en el template. Vistazo general, sin detenerte mucho.

"Del lado del frontend, en Vue 3, agregué un carrito en un panel deslizable activado por un ícono, para no saturar la pantalla principal. Al confirmar la compra, se abre el detalle de la orden en una pestaña nueva, y desde ahí se puede cambiar el estado de la orden y descargar un comprobante en PDF generado en el navegador con la librería jsPDF. También agregué una vista de 'Pedidos' que lista todas las órdenes, con filtro por cliente y paginación."

3. Publicar en la nube: MongoDB Atlas y Render (3-4 min)

3.1 Base de datos: MongoDB Atlas

QUÉ HACER:

1. Abre una pestaña nueva y entra a mongodb.com/cloud/atlas.
2. Muestra tu Organization y Project arriba, y el menú izquierdo con Clusters.
3. Clic en tu cluster (o, si crearas uno desde cero, en "+ Create" → plan M0 Free).
4. Security → Database Access: muestra el usuario de base de datos (sin mostrar la contraseña).
5. Security → Network Access: muestra la entrada 0.0.0.0/0 (permitir cualquier IP, necesaria para Render).
6. Clusters → Connect → Drivers → .NET: muestra el connection string (contraseña tapada).

"La base de datos está alojada en MongoDB Atlas, un servicio de base de datos documental administrado en la nube. Aquí está el clúster, con un usuario de base de datos dedicado y, en la sección de Network Access, la regla que permite conexiones desde cualquier IP — necesaria porque Render no tiene una IP fija en el plan gratuito. El connection string que se genera aquí —oculto la contraseña por seguridad— es lo único que Order.API necesita para conectarse; no se guarda en el código fuente, sino como variable de entorno en el siguiente paso."

3.2 Microservicio: Render

QUÉ HACER:

1. Abre otra pestaña y entra a dashboard.render.com.
2. Muestra los servicios ya desplegados: catalog-api, basket-api, order-api.
3. Clic en order-api.
4. Pestaña Settings: señala Dockerfile Path (`src/Order.API/Dockerfile`) y el repo de GitHub conectado.
5. Pestaña Environment: muestra la lista de variables configuradas (sin revelar contraseñas):
ASPNETCORE_ENVIRONMENT, MongoDB__ConnectionString, MongoDB__DatabaseName,
MongoDb__CollectionName, Services__BasketApi, Services__CatalogApi, Cors__AllowedOrigins__0.
6. Pestaña Logs/Events: muestra el último deploy terminando en "Live".
7. Muestra la URL pública arriba del panel.

"El microservicio se publica en Render, el mismo servicio en la nube que ya usábamos para Catalog.API y Basket.API — así que el proceso fue el mismo: crear un nuevo 'Web Service' apuntando al mismo repositorio de GitHub, indicando la ruta de su Dockerfile específico, y configurando sus propias variables de entorno. Lo único distinto aquí es que, además del connection string de MongoDB, Order.API necesita saber dónde están los otros dos microservicios: las variables Services__BasketApi y Services__CatalogApi apuntan a sus URLs públicas en Render, para que la comunicación entre microservicios funcione también en producción, no solo en local. Cada git push al repositorio redespiega automáticamente los servicios que hayan cambiado, sin pasos manuales."

4. Demo completa en producción (7-8 min)

4.1 Agregar al carrito y comprar

QUÉ HACER:

1. Cambia a la pestaña de Netlify (ya debería tener 1 producto en el carrito).
2. Clic en el ícono  arriba a la derecha para abrir el panel.
3. Clic en el botón azul "Realizar compra".
4. Se abre una pestaña nueva — cámbiate a ella.

"Ahora una demostración completa, todo contra el entorno de producción: frontend en Netlify, y los tres microservicios —Catalog, Basket y Order— desplegados en Render, cada uno con su propia base de datos en la nube. Al hacer clic en 'Realizar compra', el frontend llama a `POST /api/orders` con el `customerId`, el `basketId`, y un `Idempotency-Key` generado automáticamente. La orden se abre en una pestaña nueva, mostrando el identificador único, la fecha, la hora, los productos, y el estado inicial: `Pending`."

4.2 Verificar en MongoDB Atlas

QUÉ HACER:

1. Copia el ID de la orden en pantalla.
2. Cambia a la pestaña de MongoDB Atlas.
3. Menú izquierdo → "Clusters" → botón "Browse Collections" de tu cluster.
4. En la lista de bases, clic en `OrdersDb` → colección `orders`.
5. Busca el documento con ese ID (o el más reciente) y haz clic para expandirlo.

"Para comprobar que la persistencia es real y no solo en memoria, entro a MongoDB Atlas y busco el documento con el mismo identificador que acabamos de ver — aquí está, con exactamente los mismos datos: cliente, items, subtotal, impuesto, total y estado."

4.3 Cambiar el estado y descargar PDF

QUÉ HACER:

1. Regresa a la pestaña del detalle de la orden.
2. Clic en "Confirmar orden" — observa el badge cambiar de amarillo a verde.
3. Clic en  "Descargar PDF".
4. Abre el PDF descargado y muéstralo brevemente.

"Cambio el estado de la orden de `Pending` a `Confirmed`, usando el endpoint `PATCH /api/orders/{id}/status`, que valida que la transición sea permitida. Y descargo el comprobante en PDF, generado del lado del cliente con fecha, hora, productos, cantidades, precios y total."

4.4 Vista de Pedidos

QUÉ HACER:

1. Regresa a Netlify, catálogo principal.
2. Clic en  "Pedidos" (se abre pestaña nueva).
3. Muestra la tabla completa con scroll.
4. Escribe un nombre de cliente en "Filtrar por cliente" → clic "Buscar".

5. Clic en "Ver todas" para regresar al listado completo.

"En la vista de Pedidos se listan todas las órdenes generadas, con paginación, y puedo filtrar por cliente específico para ver solo sus compras. Cada fila muestra el cliente, la fecha y hora exactas, los productos, el total y el estado con un color distinto según si está pendiente, confirmada o cancelada; y haciendo clic en cualquier parte de la fila se llega al mismo detalle que vimos antes."

4.5 Cierre de la demo: los tres servicios trabajando juntos

QUÉ HACER: Vuelve a Netlify, catálogo principal. Abre brevemente el panel del carrito y ciérralo, solo para dejar claro que sigue disponible.

"Con esto se completa el flujo de punta a punta: el catálogo y el carrito, que ya existían de la primera fase del proyecto, ahora se conectan con este nuevo microservicio de órdenes sin que se haya tocado su lógica interna — Order.API simplemente los consume por HTTP, como un cliente más. Esa es la ventaja de la arquitectura de microservicios: cada servicio se puede extender o reemplazar sin afectar a los demás, mientras se respeta el contrato de su API."

5. Cierre (30-40 s)

QUÉ HACER: Vuelve al editor o a Netlify en pantalla principal.

"En resumen: Order.API es un microservicio independiente con Minimal API, persistencia en MongoDB Atlas, idempotencia mediante `Idempotency-Key`, control de estados con transiciones validadas, e integración HTTP con los microservicios existentes de catálogo y carrito. Reutiliza los mismos patrones de diseño del resto del proyecto — manejo de errores centralizado, separación por capas, variables de entorno para toda la configuración sensible— y quedó publicado en la nube, funcionando en conjunto con el frontend en un flujo de compra completo y verificable de principio a fin. Gracias."

Checklist antes de grabar

- ☐ Las 2 pestañas del navegador abiertas y listas (Netlify, MongoDB Atlas).
- ☐ Los 6 archivos del editor abiertos en pestañas separadas.
- ☐ Los 3 servicios en Render respondiendo.
- ☐ MongoDB Atlas con el cluster activo (no pausado — revisa antes, si dice "paused" dale Resume).
- ☐ Un producto ya agregado al carrito en Netlify antes de grabar.
- ☐ Contraseñas/connection strings tapadas si el video se comparte publicamente.